

MUSTAFACODE

Lecture 08

Member Functions, Constructors & Copy Concepts (C++)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 8: Member Functions, Constructors & Copy Concepts (C++)

Chapter 08 — Member Functions, Constructors & Copy Concepts (C++)

◆ 08.1 Member Functions

Member functions wo functions hotay hain jo **class ke data (variables)** par kaam karte hain.

Key Points:

- Class ke andar jo functions hotay hain unhein member functions kehte hain
- Public member functions class ka **interface** hotay hain (bahar se access inhi ke through hota hai)

Real Example:

```
class BankAccount {  
    float balance;  
  
public:  
    void deposit(float amount) {  
        balance += amount;  
    }  
};
```

👉 Yahan `deposit()` member function hai jo balance modify karta hai.

◆ 08.2 Defining Member Functions

Member functions 2 tareeqon se define hotay hain:

✓ (a) Inside Class

```
class Student {  
    int rollNo;  
  
public:  
    void setRollNo(int r) {  
        rollNo = r;  
    }  
};
```

👉 Simple aur inline hota hai

✓ (b) Outside Class

```
class Student {  
    int rollNo;  
  
public:  
    void setRollNo(int r);  
};  
  
void Student::setRollNo(int r) {  
    rollNo = r;  
}
```

👉 :: scope resolution operator batata hai ke function kis class ka hai

💡 Real Example:

```
class Car {  
public:  
    void start();  
};  
  
void Car::start() {  
    cout << "Car started";  
}
```

◆ 08.3 Inline Functions

📌 Definition:

Inline functions wo hotay hain jahan function call ki jagah **code directly copy ho jata hai**

📌 Key Points:

- Function call overhead reduce hota hai
- Small functions ke liye best
- Compiler zaroori nahi inline kare

✅ Inside Class (Automatically Inline)

```
class Student {  
public:  
    void show() {  
        cout << "Hello";  
    }  
};
```

✅ Outside Class Inline

```
class Student {  
public:  
    void show();  
};  
  
inline void Student::show() {  
    cout << "Hello";  
}
```

💡 Real Example:

```
inline int square(int x) {  
    return x * x;  
}
```

◆ 08.4 Constructor

📌 Definition:

Constructor ek special function hota hai jo **object ban'tay hi automatically run hota hai**.

📌 Properties:

- Same name as class
- No return type
- Automatically called

💡 Example:

```
class Student {
    int rollNo;

public:
    Student() {
        rollNo = 0;
    }
};
```

💡 Real Example:

```
class Phone {
public:
    Phone() {
        cout << "Phone created";
    }
};
```

◆ 08.5 Constructor Properties

- Public hota hai
 - Initialization ke liye use hota hai
 - Object creation ke waqt automatically call hota hai
-

◆ 08.6 Default Constructor

📌 Definition:

Constructor without parameters OR all default parameters

💡 Example:

```
class Student {  
    int rollNo;  
    float GPA;  
  
public:  
    Student() {  
        rollNo = 0;  
        GPA = 0.0;  
    }  
};
```

⚠ Rule:

Agar koi constructor define kar dein to compiler default constructor nahi banata.

◆ 08.7 Constructor Overloading

📌 Definition:

Same class mein multiple constructors with different parameters

💡 Example:

```
class Student {  
public:  
    Student() {}  
    Student(char *name) {}  
    Student(char *name, int rollNo) {}  
};
```

◆ 08.8 Constructor with Default Arguments

📌 Definition:

Ek hi constructor multiple constructors ka kaam karta hai using default values

💡 Example:

```
class Student {  
public:  
    Student(char *name = NULL, int rollNo = 0, float GPA = 0.0) {}  
};
```

💡 Usage:

```
Student s1;  
Student s2("Ali");  
Student s3("Ali", 1);  
Student s4("Ali", 1, 3.5);
```

◆ 08.9 Copy Constructor

📌 Definition:

Copy constructor ek object ko dusre object se initialize karta hai

📌 Syntax:

```
Student(const Student &obj)
```

💡 Example:

```
class Student {  
public:  
    int rollNo;  
  
    Student(const Student &obj) {  
        rollNo = obj.rollNo;  
    }  
};
```

💡 Real Example:

```
Student a;  
Student b = a;
```

◆ 08.10 Shallow Copy

📌 Definition:

Jab object copy hota hai lekin **same memory shared hoti hai**

⚠️ Problems:

- Dangling pointer
- Data accidental change
- Memory corruption

◆ 08.11 Deep Copy

📌 Definition:

Jab object copy hota hai aur **new memory allocate hoti hai**

Example:

```
Student::Student(const Student &obj) {  
    name = new char[strlen(obj.name) + 1];  
    strcpy(name, obj.name);  
}
```

Final Comparison

Concept	Description
Inline Function	Fast execution, code copy hota hai
Constructor	Object creation time initialization
Default Constructor	No parameter constructor
Overloading	Multiple constructors
Copy Constructor	Object to object copy
Shallow Copy	Shared memory
Deep Copy	Separate memory

Final Summary

- Member functions = class behavior
- Constructors = object initialization
- Inline = performance optimization
- Copy constructor = object duplication
- Shallow copy = risky shared memory
- Deep copy = safe independent memory



Exam Tip:

👉 Always remember:

- Dynamic memory = Deep copy zaroori
- Simple classes = default shallow copy fine



MUSTAFA CO